ADPro pour politiques de diffusion robotiques

Explication des étapes et implémentation dans le code

Projet SIMUL – Franka Panda

17 mars 2026

1 Objectif

ADPro (*Adaptive Diffusion Policy*) est une adaptation *test-time* d'une politique de diffusion pré-entraînée. L'idée est de ne pas réentraîner le modèle, mais de guider le débruitage avec de la géométrie de tâche.

Dans ce projet, ADPro est implémenté dans `models/adpro.py` sous la classe `ADPro`, avec deux variantes :

- `implementation='paper'` : version la plus fidèle au papier (FGR + guidance Chamfer + contrainte sphérique).
- `implementation='practical'` : version plus robuste/pragmatique en simulation (target explicite + blend optionnel).

2 Notation

- Observation : $O = (O^0, O^1)$ avec O^0 nuage de points pince et O^1 nuage de points scène.
- Action 7D : $a = [x, y, z, q_x, q_y, q_z, q_w]$.
- Étape de diffusion : t (du bruit fort vers faible).
- Politique de base : réseau $\epsilon_\theta(O, a_t, t)$.

3 Étape 1 – Initialisation consciente de la tâche

3.1 Principe théorique

Au lieu de partir d'un bruit gaussien pur $a_T \sim \mathcal{N}(0, I)$, ADPro calcule une pose initiale structurée a_0^{est} via la géométrie de scène, puis la projette au niveau de bruit $t = M$:

$$a_M = \sqrt{\bar{\alpha}_M}\, a_0^{\text{est}} + \sqrt{1 - \bar{\alpha}_M}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{1}$$

3.2 Implémentation

Fonction : `_task_aware_init(obs, t_start)`.

- Sous-échantillonnage des points : `subsample_pointcloud`.
- Filtrage des points objet (pour réduire l'effet table) : masque en z.
- Variante `paper` : estimation rigide complète par `fast_global_registration` puis conversion via `se3_to_action`.
- Variante `practical` : cible explicite de grasp (via `_target_grasp_action`) + correction par centroïdes.

4 Étape 2 – Un pas de débruitage diffusion

4.1 Principe

On exécute d'abord le pas DDPM standard pour obtenir $\tilde{a}_{t-1}$.

4.2 Implémentation

Dans `_adpro_step` :
— Prédiction du bruit : `eps_pred = policy.network.forward(obs, a_t, t)`.
— Pas DDPM : `a_tilde = policy.ddpm_step(...)`.
— Option déterministe : `deterministic_denoising=True` coupe le bruit de reverse-process pour plus de stabilité.

5 Étape 3 – Estimation de l'action propre (formule type Tweedie)

$$\hat{a}_0 = \frac{a_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(O, a_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{2}$$

Implémentation : `_predict_clean_action`.

6 Étape 4 – Guidance sur manifold de tâche

6.1 Version papier (surface-à-surface)

On construit la perte géométrique sur nuages de points (Chamfer) :

$$\mathcal{L} = d_{\text{Chamfer}}\big(T(\hat{a}_0)\,O^0,\ O^1\big). \tag{3}$$

Puis on corrige l'action intermédiaire dans la direction opposée au gradient.

Implémentation dans `_apply_task_manifold_guidance` :
— Conversion action → pose : `action_to_se3`.
— Remise en repère local pince puis application de $T(\hat{a}_0)$ (cohérence des repères).
— Gradient Chamfer : `chamfer_distance_grad`.

6.2 Version pratique (point-à-point)

La guidance pousse vers une `target_grasp_action` explicite (position + quaternion cible) via une erreur L2 simplifiée.

7 Étape 5 – Contrainte sphérique (Eq. 9)

La correction est normalisée puis mise à l'échelle par le bruit courant :

$$a_{t-1} = \tilde{a}_{t-1} - s\,\sqrt{d}\,\sigma_t\,\frac{\nabla_a \mathcal{L}}{\|\nabla_a \mathcal{L}\|}, \tag{4}$$

où d est la dimension d'action et s est un facteur de stabilisation (dans le code : `spherical_scale`).

Implémentation :
— `use_spherical=True` : active la formule normalisée.
— `spherical_scale` : contrôle l'intensité du pas.

8 Étape 6 – Boucle complète d'inférence

Dans `inference(...)` :

1. Choix de l'initialisation : task-aware (si `use_fgr`) ou gaussienne.
2. Boucle décroissante en t de `t_start` vers 0.
3. À chaque itération : DDPM → estimation $\hat{a}_0$ → guidance manifold → contrainte sphérique.
4. Sortie finale : action a_0 appliquée à l'environnement.

9 Paramètres importants et rôle

— `implementation` : `paper` vs `practical`.
— `M` : niveau de bruit initial (plus petit = init plus proche de l'action finale).
— `spherical_scale` : stabilité vs agressivité de la guidance.
— `deterministic_denoising` : stabilité temporelle en simulation.
— `use_blend` (practical) : stabilisation additionnelle vers la cible explicite.

10 Lien avec la visualisation

Dans `simulate_realtime.py`, l'option `-show-denoise` permet de visualiser :
— la trajectoire interne $a_t \to a_0$ (baseline vs ADPro),
— l'initialisation ADPro,
— les métriques de guidance (norme gradient, norme de pas, blend).

11 Résumé

ADPro améliore une politique de diffusion en injectant de la structure géométrique au test-time : initialisation orientée tâche, guidance manifold, et normalisation sphérique des mises à jour. La variante `paper` maximise la fidélité théorique ; la variante `practical` maximise la robustesse en simulation.